```
-- ============================================================
-- Database Schema for RaceDay Event Management System
-- ============================================================

-- Drop tables if they already exist (to allow clean execution)
DROP TABLE IF EXISTS results CASCADE;
DROP TABLE IF EXISTS registrations CASCADE;
DROP TABLE IF EXISTS routes CASCADE;
DROP TABLE IF EXISTS categories CASCADE;
DROP TABLE IF EXISTS events CASCADE;
DROP TABLE IF EXISTS users CASCADE;

-- 1. USERS TABLE
CREATE TABLE users (
    user_id SERIAL PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    phone_number VARCHAR(15),
    user_role VARCHAR(20) NOT NULL CHECK (user_role IN ('Organiser',
'Participant', 'Admin')),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

```
-- 2. EVENTS TABLE
CREATE TABLE events (
    event_id SERIAL PRIMARY KEY,
    organiser_id INT NOT NULL,
    event_name VARCHAR(150) NOT NULL,
    event_type VARCHAR(50) NOT NULL CHECK (event_type IN ('Running',
'Walking', 'Cycling', 'Multi-Sport')),
    location VARCHAR(100) NOT NULL,
    event_date DATE NOT NULL,
    description TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_events_organiser FOREIGN KEY (organiser_id)
        REFERENCES users(user_id) ON DELETE CASCADE
);

-- 3. CATEGORIES TABLE
CREATE TABLE categories (
    category_id SERIAL PRIMARY KEY,
    event_id INT NOT NULL,
    category_name VARCHAR(100) NOT NULL, -- e.g., "42km Full Marathon", "10km
Walk"
    distance_km DECIMAL(5, 2) NOT NULL,
    entry_fee DECIMAL(10, 2) NOT NULL,
    max_participants INT,
    start_time TIME NOT NULL,
```

```
    CONSTRAINT fk_categories_event FOREIGN KEY (event_id)
        REFERENCES events(event_id) ON DELETE CASCADE
);

-- 4. ROUTES TABLE
CREATE TABLE routes (
    route_id SERIAL PRIMARY KEY,
    event_id INT UNIQUE NOT NULL,
    start_location VARCHAR(100) NOT NULL,
    finish_location VARCHAR(100) NOT NULL,
    elevation_gain_m INT,
    route_map_url VARCHAR(255),
    weather_forecast TEXT,
    CONSTRAINT fk_routes_event FOREIGN KEY (event_id)
        REFERENCES events(event_id) ON DELETE CASCADE
);

-- 5. REGISTRATIONS TABLE
CREATE TABLE registrations (
    registration_id SERIAL PRIMARY KEY,
    user_id INT NOT NULL,
    category_id INT NOT NULL,
    race_number VARCHAR(20),
    registration_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
```

```
    payment_status VARCHAR(20) DEFAULT 'Pending' CHECK (payment_status IN
('Pending', 'Paid', 'Cancelled')),
    CONSTRAINT fk_registrations_user FOREIGN KEY (user_id)
        REFERENCES users(user_id) ON DELETE CASCADE,
    CONSTRAINT fk_registrations_category FOREIGN KEY (category_id)
        REFERENCES categories(category_id) ON DELETE CASCADE,
    CONSTRAINT unique_user_category UNIQUE (user_id, category_id)
);


-- 6. RESULTS TABLE
CREATE TABLE results (
    result_id SERIAL PRIMARY KEY,
    registration_id INT UNIQUE NOT NULL,
    finish_time INTERVAL, -- Format: HH:MM:SS
    overall_position INT,
    category_position INT,
    status VARCHAR(20) DEFAULT 'Finished' CHECK (status IN ('Finished', 'DNF',
'DNS', 'DQ')),
    CONSTRAINT fk_results_registration FOREIGN KEY (registration_id)
        REFERENCES registrations(registration_id) ON DELETE CASCADE
);
```

The API Endpoint Plan for the **RaceDay** platform directly matches the database schema provided in Section A. You can save this Markdown table into a file named `endpoints.md` inside your repository's `/docs` folder (`/docs/endpoints.md`).

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
POST	`/api/auth/register`	Registers a new user (Organiser or Participant) in the system.	None (Public)	`{ firstName, lastName, email, password, phoneNumber, role }`	`201 Created` - User object with JWT token. `400 Bad Request` - Validation error or duplicate email.
POST	`/api/auth/login`	Authenticates a user and returns a JWT access token.	None (Public)	`{ email, password }`	`200 OK` - Auth token and user details. `401 Unauthorized` - Invalid credentials.

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
GET	`/api/users/profile`	Retrieves the profile details of the currently authenticated user.	Any (Logged in)	None	`200 OK` - Current user details. `401 Unauthorized` - Missing or invalid token.
PUT	`/api/users/profile`	Updates the profile details of the logged-in user.	Any (Logged in)	`{ firstName, lastName, phoneNumber }`	`200 OK` - Updated user profile. `400 Bad Request` - Invalid payload.
GET	`/api/events`	Fetches a list of all upcoming events with optional filtering	None (Public)	None	`200 OK` - Array of event objects.

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
		by type or location.			
GET	`/api/events/{id}`	Retrieves detailed information for a specific event including categories and route info.	None (Public)	None	`200 OK` - Event details with nested categories and route. `404 Not Found` - Event does not exist.
POST	`/api/events`	Creates a new event entry in the system.	Organiser	`{ name, type, location, date, description }`	`201 Created` - Created event object. `403 Forbidden` - Insufficient permissions.
PUT	`/api/events/{id}`	Updates details of	Organiser	`{ name, type,`	`200 OK` -

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
		an existing event managed by the organiser.		`location, date, description }`	Updated event object. `403 Forbidden` - Not event owner. `404 Not Found` - Event missing.
DELETE	`/api/events/{id}`	Removes an event and associated categories from the platform.	Organiser	None	`200 OK` - Confirmation message. `403 Forbidden` - Not event owner.
GET	`/api/events/{eventId}/categories`	Lists all race categories /distances	None (Public)	None	`200 OK` - Array of

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
		available for a given event.			category objects. `404 Not Found` - Event does not exist.
POST	`/api/events/{eventId}/categories`	Adds a new race distance/category to a specific event.	Organiser	`{ name, distanceKm, entryFee, maxParticipants, startTime }`	`201 Created` - Created category object. `403 Forbidden` - Not event owner.
POST	`/api/registrations`	Enrols the authenticated participant into a specific race category.	Participant	`{ categoryId }`	`201 Created` - Registration details with payment status.

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
					`400 Bad Request` - Event full. `409 Conflict` - User already enrolled.
GET	`/api/registrations/my-events`	Fetches all event enrolments and history for the logged-in participant.	Participant	None	`200 OK` - List of participant's active and past registrations.
DELETE	`/api/registrations/{id}`	Cancels an existing enrolment for a race category.	Participant	None	`200 OK` - Cancellation success message. `404 Not Found` - Registrat

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
					ion not found.
GET	`/api/categories/{categoryId}/results`	Retrieves overall participant finish times and standings for a race category.	None (Public)	None	`200 OK` - Array of result records with rank and finish times.
POST	`/api/categories/{categoryId}/results`	Uploads or records race results for participants in a category.	Organiser	`[{ registrationId, finishTime, overallPosition, categoryPosition, status }]`	`201 Created` - Confirmation of inserted results. `403 Forbidden` - Not event owner.
GET	`/api/events/{eventId}/route`	Retrieves route details, elevation profile, and weather forecasts for an event.	None (Public)	None	`200 OK` - Route details and weather info. `404 Not`

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
					`Found` - Route missing.

```
-- ============================================================================
-- Section C: SQL Database Script (SSMS / Microsoft SQL Server Compatible)
-- Project: RaceDay Event Management System
-- Description: DDL Schema creation with constraints and realistic seed data.
-- ============================================================================

-- ----------------------------------------------------------------------------
-- 1. DATABASE CREATION AND CLEANUP
-- ----------------------------------------------------------------------------
-- Ensure clean execution by dropping foreign keys and tables if they exist
IF OBJECT_ID('dbo.Results', 'U') IS NOT NULL DROP TABLE dbo.Results;
IF OBJECT_ID('dbo.Registrations', 'U') IS NOT NULL DROP TABLE dbo.Registrations;
IF OBJECT_ID('dbo.Routes', 'U') IS NOT NULL DROP TABLE dbo.Routes;
IF OBJECT_ID('dbo.Categories', 'U') IS NOT NULL DROP TABLE dbo.Categories;
IF OBJECT_ID('dbo.Events', 'U') IS NOT NULL DROP TABLE dbo.Events;
IF OBJECT_ID('dbo.Users', 'U') IS NOT NULL DROP TABLE dbo.Users;
GO

-- ----------------------------------------------------------------------------
-- 2. CREATE TABLES WITH CONSTRAINTS
-- ----------------------------------------------------------------------------

-- Entity 1: Users
CREATE TABLE dbo.Users (
    UserId INT IDENTITY(1,1) NOT NULL,
    FirstName NVARCHAR(50) NOT NULL,
    LastName NVARCHAR(50) NOT NULL,
    Email NVARCHAR(100) NOT NULL,
    PasswordHash NVARCHAR(255) NOT NULL,
```

```
    PhoneNumber VARCHAR(15) NULL,
    UserRole VARCHAR(20) NOT NULL,
    CreatedAt DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME(),

    CONSTRAINT PK_Users PRIMARY KEY (UserId),
    CONSTRAINT UQ_Users_Email UNIQUE (Email),
    CONSTRAINT CK_Users_UserRole CHECK (UserRole IN ('Organiser', 'Participant', 'Admin'))
);

-- Entity 2: Events
CREATE TABLE dbo.Events (
    EventId INT IDENTITY(1,1) NOT NULL,
    OrganiserId INT NOT NULL,
    EventName NVARCHAR(150) NOT NULL,
    EventType VARCHAR(50) NOT NULL,
    Location NVARCHAR(100) NOT NULL,
    EventDate DATE NOT NULL,
    Description NVARCHAR(MAX) NULL,
    CreatedAt DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME(),

    CONSTRAINT PK_Events PRIMARY KEY (EventId),
    CONSTRAINT FK_Events_Users FOREIGN KEY (OrganiserId)
        REFERENCES dbo.Users(UserId) ON DELETE CASCADE,
    CONSTRAINT CK_Events_EventType CHECK (EventType IN ('Running', 'Walking', 'Cycling', 'Multi-
Sport'))
);

-- Entity 3: Categories
CREATE TABLE dbo.Categories (
```

```
    CategoryId INT IDENTITY(1,1) NOT NULL,
    EventId INT NOT NULL,
    CategoryName NVARCHAR(100) NOT NULL,
    DistanceKm DECIMAL(6, 2) NOT NULL,
    EntryFee DECIMAL(10, 2) NOT NULL DEFAULT 0.00,
    MaxParticipants INT NULL,
    StartTime TIME NOT NULL,

    CONSTRAINT PK_Categories PRIMARY KEY (CategoryId),
    CONSTRAINT FK_Categories_Events FOREIGN KEY (EventId)
        REFERENCES dbo.Events(EventId) ON DELETE CASCADE
);

-- Entity 4: Routes
CREATE TABLE dbo.Routes (
    RouteId INT IDENTITY(1,1) NOT NULL,
    EventId INT NOT NULL,
    StartLocation NVARCHAR(100) NOT NULL,
    FinishLocation NVARCHAR(100) NOT NULL,
    ElevationGainMeters INT NULL,
    RouteMapUrl NVARCHAR(255) NULL,
    WeatherForecast NVARCHAR(255) NULL,

    CONSTRAINT PK_Routes PRIMARY KEY (RouteId),
    CONSTRAINT UQ_Routes_EventId UNIQUE (EventId),
    CONSTRAINT FK_Routes_Events FOREIGN KEY (EventId)
        REFERENCES dbo.Events(EventId) ON DELETE CASCADE
);
```

```
-- Entity 5: Registrations (Event Enrolments)
CREATE TABLE dbo.Registrations (
    RegistrationId INT IDENTITY(1,1) NOT NULL,
    UserId INT NOT NULL,
    CategoryId INT NOT NULL,
    RaceNumber VARCHAR(20) NULL,
    RegistrationDate DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME(),
    PaymentStatus VARCHAR(20) NOT NULL DEFAULT 'Pending',

    CONSTRAINT PK_Registrations PRIMARY KEY (RegistrationId),
    CONSTRAINT UQ_Registrations_UserCategory UNIQUE (UserId, CategoryId),
    CONSTRAINT FK_Registrations_Users FOREIGN KEY (UserId)
        REFERENCES dbo.Users(UserId) ON DELETE NO ACTION,
    CONSTRAINT FK_Registrations_Categories FOREIGN KEY (CategoryId)
        REFERENCES dbo.Categories(CategoryId) ON DELETE CASCADE,
    CONSTRAINT CK_Registrations_PaymentStatus CHECK (PaymentStatus IN ('Pending', 'Paid',
'Cancelled'))
);

-- Entity 6: Results
CREATE TABLE dbo.Results (
    ResultId INT IDENTITY(1,1) NOT NULL,
    RegistrationId INT NOT NULL,
    FinishTime TIME NULL,
    OverallPosition INT NULL,
    CategoryPosition INT NULL,
    Status VARCHAR(20) NOT NULL DEFAULT 'Finished',

    CONSTRAINT PK_Results PRIMARY KEY (ResultId),
```

```
    CONSTRAINT UQ_Results_RegistrationId UNIQUE (RegistrationId),
    CONSTRAINT FK_Results_Registrations FOREIGN KEY (RegistrationId)
        REFERENCES dbo.Registrations(RegistrationId) ON DELETE CASCADE,
    CONSTRAINT CK_Results_Status CHECK (Status IN ('Finished', 'DNF', 'DNS', 'DQ'))
);
GO

-- ---------------------------------------------------------------------------
-- 3. SEED DATA (Realistic South African RaceDay Context)
-- ---------------------------------------------------------------------------

-- Seed 1: Users (2 Organisers, 3 Participants)
INSERT INTO dbo.Users (FirstName, LastName, Email, PasswordHash, PhoneNumber, UserRole)
VALUES
('Adrian', 'Sithole', 'adrian@comrades.co.za', 'hashed_pwd_101', '0821234567', 'Organiser'),
('Chantal', 'van Zyl', 'chantal@capetowncycletour.co.za', 'hashed_pwd_102', '0839876543', 'Organiser'),
('Sipho', 'Dlamini', 'sipho.dlamini@gmail.com', 'hashed_pwd_103', '0711122334', 'Participant'),
('Lize', 'Muller', 'lize.muller@yahoo.com', 'hashed_pwd_104', '0724455667', 'Participant'),
('Kabelo', 'Mokoena', 'kabelo.m@outlook.com', 'hashed_pwd_105', '0843322110', 'Participant');

-- Seed 2: Events (3 iconic South African events)
INSERT INTO dbo.Events (OrganiserId, EventName, EventType, Location, EventDate, Description)
VALUES
(1, 'Comrades Marathon', 'Running', 'Pietermaritzburg to Durban', '2027-06-13', 'The world premier ultimate human race distance road running marathon.'),
(2, 'Cape Town Cycle Tour', 'Cycling', 'Cape Peninsula', '2027-03-14', 'The iconic cycle tour around the breathtaking Cape Peninsula.'),
(1, 'Soweto Marathon', 'Running', 'Soweto, Johannesburg', '2026-11-01', 'The People Marathon celebrating the historic heritage of Soweto.');
```

-- Seed 3: Categories (Multiple distances per event)

INSERT INTO dbo.Categories (EventId, CategoryName, DistanceKm, EntryFee, MaxParticipants, StartTime)

VALUES

(1, 'Ultimate Human Race Down-Run', 89.00, 750.00, 20000, '05:30:00'),

(2, 'Classic Cape Peninsula Tour', 109.00, 650.00, 35000, '06:15:00'),

(2, 'Short Route Challenge', 42.00, 350.00, 5000, '08:00:00'),

(3, 'Full Marathon', 42.20, 320.00, 10000, '06:00:00'),

(3, 'Half Marathon', 21.10, 220.00, 12000, '06:30:00'),

(3, '10km Community Walk/Run', 10.00, 150.00, 8000, '07:00:00');

-- Seed 4: Routes

INSERT INTO dbo.Routes (EventId, StartLocation, FinishLocation, ElevationGainMeters, RouteMapUrl, WeatherForecast)

VALUES

(1, 'Pietermaritzburg City Hall', 'Moses Mabhida Stadium, Durban', 1150, 'https://raceday.co.za/maps/comrades-down.pdf', 'Sunny with moderate coastal breeze, 18°C - 26°C'),

(2, 'Grand Parade, Cape Town', 'Green Point, Cape Town', 1220, 'https://raceday.co.za/maps/ct-cycle-109.pdf', 'Clear skies, strong winds expected around Chapman Peak, 21°C'),

(3, 'FNB Stadium, Soweto', 'FNB Stadium, Soweto', 450, 'https://raceday.co.za/maps/soweto-marathon.pdf', 'Warm and clear, 15°C - 28°C');

-- Seed 5: Registrations (Sample Enrolments)

INSERT INTO dbo.Registrations (UserId, CategoryId, RaceNumber, PaymentStatus)

VALUES

(3, 1, 'BIB-10492', 'Paid'),

(4, 2, 'BIB-88210', 'Paid'),

(5, 4, 'BIB-33019', 'Paid'),

(3, 5, 'BIB-55102', 'Paid'),

(4, 6, 'BIB-00412', 'Pending');

```
-- Seed 6: Sample Results
INSERT INTO dbo.Results (RegistrationId, FinishTime, OverallPosition, CategoryPosition, Status)
VALUES
(1, '06:45:12', 1420, 310, 'Finished'),
(2, '03:15:40', 850, 112, 'Finished'),
(3, '03:42:05', 402, 85, 'Finished');
GO


-- ---------------------------------------------------------------------------
-- 4. VERIFICATION QUERY
-- ---------------------------------------------------------------------------
SELECT
    r.RegistrationId,
    CONCAT(u.FirstName, ' ', u.LastName) AS ParticipantName,
    e.EventName,
    c.CategoryName,
    r.RaceNumber,
    r.PaymentStatus,
    res.FinishTime,
    res.OverallPosition
FROM dbo.Registrations r
INNER JOIN dbo.Users u ON r.UserId = u.UserId
INNER JOIN dbo.Categories c ON r.CategoryId = c.CategoryId
INNER JOIN dbo.Events e ON c.EventId = e.EventId
LEFT JOIN dbo.Results res ON r.RegistrationId = res.RegistrationId;
GO
```